Snapshot of October 29, 2026. This dossier describes the four repositories as they stood that day; the projects have moved on since, and the repositories are the authority. Underlined links lead to the sources, frozen at their state of that day. What came next lives on gnu6dotlive.github.io.

GLOSSARY

Three words to defuse

"Open source", "repository", "proof". Three words that come up everywhere and often mean less — or more — than you think.

Open source: the recipe, not the dish

THE ANALOGY

A restaurant can publish its full recipe. That does not mean the dish is good, that the kitchen is open, or that anyone will serve you at a table. It means that **you can make it yourself**, fix it, and check what is in it.

That is exactly what a free licence does: it gives you the right to read, modify and redistribute. It promises neither quality, nor support, nor a user manual.

A repository is a folder with a memory

A "repository" is a folder of files whose history is kept. Each recorded change is called a **commit**: it keeps the date, the author and an explanation. So you can tell who changed what, when, and why.

The `README.md` file is the front door: what the author wants you to know first. The `docs/` folders hold the documentation, `evidence/` the evidence, `crates/` the code split into packages.

Commit

A recorded change, with its message and its date. The unit of a project's history.

Issue

A problem or a request, discussed in public. A visible to-do list.

Pull request

A proposed change, which the author can review before accepting it.

Licence

The contract that says what you may do with the code. MIT is permissive; GPL requires you to share your modifications.

CI

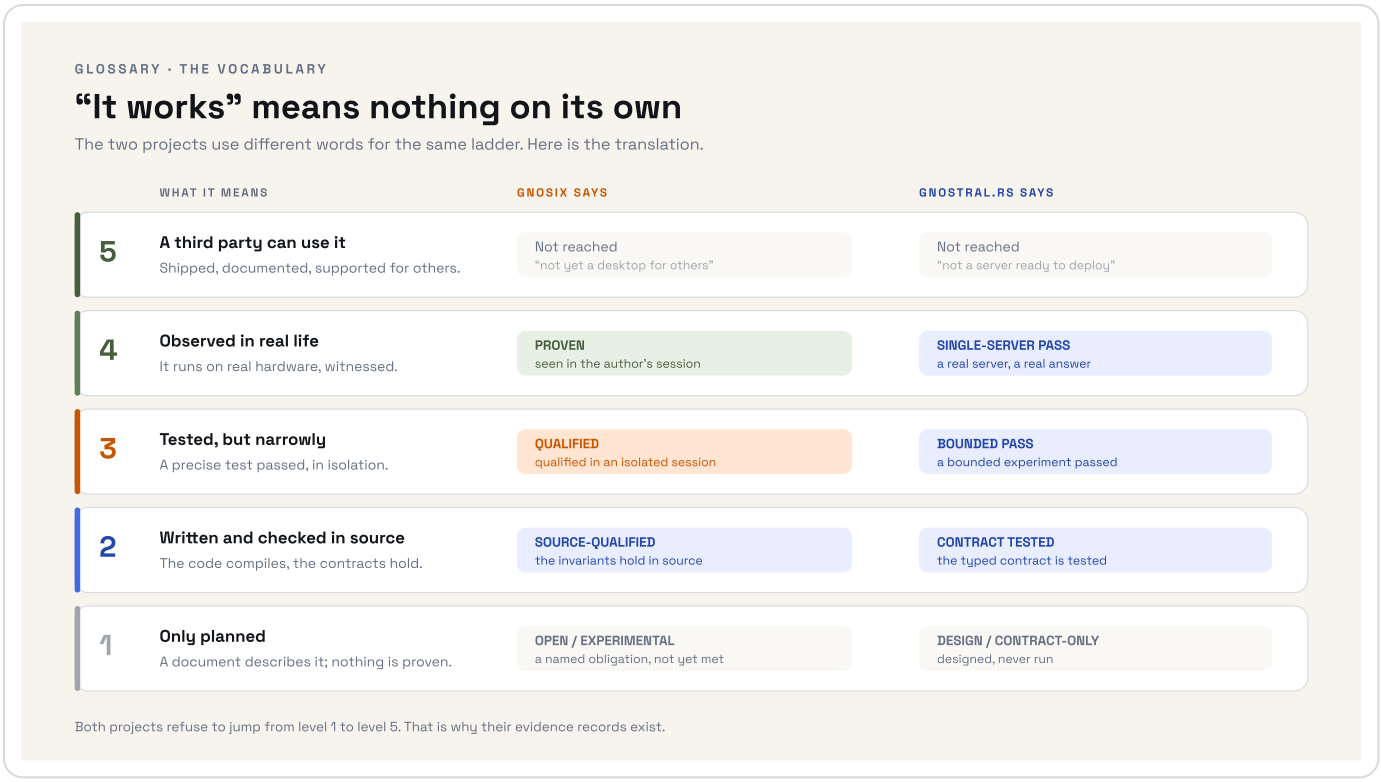
"Continuous integration": automatic checks run on every change.

Fork

An independent copy of a project, developed separately.

"It works" means nothing on its own

This is the central lesson of the four projects. A feature can be described, written, tested, observed on its author's machine, or used by a third party. Those are five different things, and confusing them is the source of almost every disappointment.



The same ladder, two vocabularies. gnosix speaks of PROVEN, QUALIFIED, SOURCE-QUALIFIED. gnostral.rs speaks of BOUNDED PASS, CONTRACT TESTED, SINGLE-SERVER PASS. Under the words, the same question: how far does the evidence go?

The vocabulary, translated

The desktop

TERM	IN PLAIN WORDS
Compositor	The program that puts the windows and the display together. On Linux, Hyprland is one.
Wayland	The modern protocol through which a program talks to the display.
GPUI	The rendering library used by gnosix, coming from the Zed editor.
Surface	An area drawn on screen: the bar, the deck, a window, a panel.
Crate	A package of Rust code. gnosix has about forty, split by domain.
Rust	A programming language known for its memory safety and performance.
Zig	Another language, used by Calix's engine.

Local artificial intelligence

TERM	IN PLAIN WORDS
Model	A very large set of numbers tuned to produce text. "30 billion parameters" gives an idea of its size.
Parameter	One of the model's internal numbers. The more there are, the bigger the model and the costlier it is to run.
Quantisation	Compressing the numbers onto fewer bits (Q2_K, Q3_K…). Less memory, slightly less precision.
MoE	"Mixture of experts": the model activates only part of its networks for each word.
Token	A fragment of a word. Throughput is measured in tokens per second (tok/s).
Inference	Using a model that has already been trained, as opposed to training it.
GPU / VRAM	The graphics card and its dedicated memory. Fast, but small on a personal machine.
Residency	Where a piece of the model is actually loaded: graphics card or main memory.
Benchmark	A comparative measurement, with a protocol. Without a protocol, a figure is just an anecdote.
mistral.rs	An inference engine written in Rust. Not to be confused with *Mistral AI*, the company that makes models.
llama.cpp	A very widespread inference engine, often used as a performance reference.

TERM	IN PLAIN WORDS
Paired A/B	Comparing two versions by alternating runs, on the same machine and the same inputs. It is what makes a "×2" credible.

Software and the scene

TERM	IN PLAIN WORDS
Package	A piece of software ready to install, with its dependencies. On Arch Linux, it is installed with `pacman`.
ALPM	The library doing the real work behind `pacman`: resolving dependencies, writing the files.
AUR	Arch's community repositories: build recipes, maintained by volunteers.
Flatpak / AppImage	Two ways to carry an application with its dependencies, outside the system. Sandboxed for one, self-contained in a single file for the other.
polkit	The mechanism that asks for authorisation at the moment of the action, for one precise operation — instead of handing out all rights in advance.
Provenance	Where a piece of software comes from: which repository, which maintainer, and so whom you will be dealing with. It is not decoration.
Scene	The set of objects, relations, positions and constraints that an engine holds to be true.
Projection / renderer	The translation of a scene into pixels, done by a rendering program. The renderer observes; it does not decide what exists.
Topology	The map of places and links — including when it changes. The common thread of the four repositories.

Four questions to ask any repository

1. What is the latest stable version?

If the answer is "none", that is not a defect: it is information. An openly experimental project is better than an unstable one presenting itself as finished.

2. What does the status document say?

Good projects have a file listing what works, what does not, and what is left to do. Look for it before believing the README.

3. Are the failures documented?

A project that publishes only its successes is hiding something. The four repositories presented here keep their failed runs, with their cause.

4. Can I check for myself?

Test commands, file fingerprints, measurement receipts: the evidence must be replayable, not just told.

Why this insistence on evidence

WHAT IT CHANGES

In a field where announcements move faster than results, the most useful thing a project can offer is not a promise: it is a **clear boundary** between what is demonstrated and what is not.

That boundary lets you read a repository without being fooled, try it without the wrong expectations, and fund it without buying an illusion.